UNIVERSITÉ CÔTE D'AZUR | MASTER 2 – SLCD

# Big Data and NoSQL

## Project 2 – BookScope

A Connected Book Discovery Platform

Hajer Akid | Academic year 2026–2027

| | |
|---|---|
| **Groups** | Two students. All members must be able to explain any part of the submitted work. |
| **Submission** | **Friday, 6 November 2026** |
| **Defence** | **Tuesday, 10 November 2026, at 2:00 p.m.** |
| **Deliverables** | A **written report** (at most 10 pages, excluding appendices); the **source code**; a **15-minute oral presentation** including a live demonstration, followed by a **10-minute question-and-answer session** with a live challenge. |

## 1. Project scenario

A digital library wants to improve how readers discover books beyond keyword search and overall popularity rankings. Its catalogue contains titles, authors and thematic labels, while historical activity records reveal which books were rated or added to reading lists. The library wants to understand connections between readers and titles, suggest books that a reader has not yet rated, and explain why particular results were returned.

You are commissioned to develop **BookScope**, a working platform that supports connected exploration of the catalogue, reader-based discovery and evidence-backed recommendations. The platform must handle highly popular books and very active readers without making less common interests invisible, and every result must be traceable to the records that support it.

BookScope will first run on a **single local data server**. You will then deploy it on **several data-serving instances** running on the same workstation and investigate the effect of distributing its data on queries that follow connections across the catalogue. Both versions must expose the same functionality and produce identical results.

## 2. Data

Use the **Goodbooks-10k** dataset, compiled from real book and reader activity. It contains metadata for 10,000 books, approximately six million ratings (integer scores from 1 to 5) by 53,424 anonymised readers, thematic tags and records of books marked for later reading.

| Resource | Dataset source |
|---|---|
| Dataset and documentation | https://github.com/zygmuntz/goodbooks-10k |
| Downloadable archives | https://github.com/zygmuntz/goodbooks-10k/releases |

The source files contain book metadata, ratings, reading lists, book–tag counts and tag names (`books.csv`, `ratings.csv`, `to_read.csv`, `book_tags.csv`, `tags.csv`). Read the documentation before joining these files: `book_id` and `goodreads_book_id` are distinct identifiers, and tag counts are aggregates that do *not* identify individual readers. Book titles are not unique identifiers. Ratings are aggregated over all editions of a book. The files contain no dates; according to the documentation,

the rows of `ratings.csv` and `to_read.csv` are sorted by time, so row order may be used as a relative order (for instance to hold out each reader's most recent ratings), but no date or duration can be derived from it.

### Selection and registration

The implemented platform must use **at least one million genuine reader–book ratings** together with the corresponding book metadata, reading lists and tags. Define a **selection rule** (for instance a sampling rule over readers, a group of thematic interests, or a combination of both) and explain the consequences of selecting a subset for the connections that remain visible.

Each group registers its selection rule with the instructor before starting the implementation. **No two groups may register the same selection**; the instructor may ask a group to modify a selection that overlaps too much with another one. The instructor will use the registered selections to prepare the defence challenges (Section 6). Before building anything, set aside two disjoint sets of real ratings from your selection, and document both: an **update batch** for service F6 and an **evaluation set** for experiment E3.

### Readers with short histories

Every reader in the dataset has at least 19 ratings, and no reader has more than 200. To study readers with short histories, you may truncate a genuine reader's history to its $k$ earliest ratings, using the documented time order; truncated histories must be labelled as such and never counted as additional readers.

### Quality

Report the number of readers, books, ratings, reading-list entries and tags retained, and describe the distribution of activity per reader and per book. In an appendix, document **at least five data issues actually present in your subset** (for example: missing identifiers or metadata, references that cannot be resolved, duplicate records, ambiguous titles, tags that describe reading status rather than content), each with its frequency and at least one concrete identifier. These examples will be checked against the source files.

Use **real records only** for the application and its experiments. Do not create artificial readers, ratings or connections, or duplicate records to increase the reported dataset size.

## 3. System to develop

Develop the following services through a usable interface (web interface, API or command-line application). All inputs must be supplied at execution time, and results must be computed from the stored data for previously unseen requests. Precomputed relationships, similarity scores or aggregates are allowed if the system maintains them and F6 keeps them up to date; answers prepared in advance for fixed examples, or tables assembled by hand, are not.

**F1. Explore a book and its connections.** Given a title or book identifier, retrieve its catalogue information and identify related books using both thematic information and patterns in reader evaluations. When a title matches several books, the ambiguity must be shown, not resolved silently. Explain the evidence supporting each connection, distinguishing a genuine shared readership from a link caused solely by an extremely common tag.

**F2. Compare reader profiles.** Given two reader identifiers, identify meaningful similarities and disagreements in their evaluations. Return the books supporting the comparison and a measure of the strength of the connection. The result must account for readers with very different activity

levels and for books rated by unusually large numbers of people, and must state explicitly when the two readers share too few books for a meaningful comparison.

**F3. Recommend books to a reader.** Produce a ranked list of books that a selected reader has not rated. Recommendations must use connections present in the activity and catalogue data, provide an understandable explanation for each result, and avoid simply reproducing the overall popularity ranking. Define how readers with very few ratings (including truncated histories) are handled. Quantify how far the recommendations depart from popularity, for instance with the share of recommended books among the 100 most rated books or their median popularity.

**F4. Trace a connection between books.** Given two books that do not necessarily share an author or tag, find and explain a meaningful indirect connection through observed reader activity and, where justified, catalogue attributes. Distinguish alternative connections, limit uninformative paths through highly connected books or readers, and explain when no convincing connection can be established.

**F5. Discover links between reading interests.** Given two selected thematic interests, identify books or reader groups that connect them. Rank the findings using explicit evidence from the dataset rather than the number of generic tags alone. Show whether a strong apparent connection persists when very popular books and highly active readers are discounted.

**F6. Update reader activity.** Import the update batch without losing previous data or creating duplicates. Define the identity of a rating and a justified policy for a pair (reader, book) that already exists with a *different* score. Importing the same batch twice must leave the stored data unchanged. The source files contain no conflicting duplicate ratings: the policy must be implemented, but testing it on real conflicting records is optional. The update must work on both deployments, and the services above must reflect the new information.

For each service, document the accepted inputs and returned outputs and provide one example extracted from the real dataset. Any recommendation or reported connection must be traceable: given a result, the system must be able to list the underlying records (reader, book, rating, tag counts) that support it.

## 4. Implementation and distribution

### Part I – Local prototype

Design the representation of catalogue information and reader activity, then implement all six services using one local data server, that is, a database system running as a separate server process queried by the application through a network endpoint (an embedded library such as SQLite or DuckDB qualifies only if it is wrapped in such a server process). Document the chosen data-management technology, the relationships and attributes retained, the intended access patterns and the measures taken to support frequent operations efficiently.

Use actual measurements to justify at least one significant modelling trade-off. Implement and measure **one alternative representation** (data model, precomputed relationships or indexing) on the same data and on at least two representative requests, and explain why it was retained or rejected. The preparation and loading procedure must be reproducible.

### Part II – Distributed prototype

Implement a second deployment with **at least two separate data-serving instances**, running as processes or containers with distinct endpoints and independent storage. A data-serving instance is a database or storage server that stores part of the data and answers queries on them; a processing engine alone (for instance Spark reading files) does not count as one. The reader-activity records

must be distributed across the instances, rather than merely copied in their entirety; catalogue data (books, tags) may be replicated on every instance. BookScope must retain a single coherent interface for its users.

Define a placement strategy based on the connections and access patterns of the application. Show how the distribution affects a reader-centred request, a book-centred request and one operation traversing several reader–book connections. For each, identify which instances are contacted, which work is completed locally, and what is exchanged between instances. Verify that both deployments produce **identical results** on identical records and inputs, using an automated comparison. Compare results after a canonical ordering, with ties broken deterministically. The comparison must include requests involving readers and books stored on different instances.

Evaluate **one alternative distributed configuration**, changing the placement strategy or the number of instances.

**Losing an instance.** Stop one data-serving instance and run F2 on two readers whose ratings are partly stored on that instance, then F3. Show what the user receives. A partial answer must never be presented as complete: a recommendation computed from part of a reader's history must say so, or the request must fail explicitly. Distinguish what your implementation actually recovers from what a replicated production system could recover. A multi-process deployment on one workstation must not be presented as equivalent to a cluster of independent machines.

## 5. Experimental investigation

The library anticipates two contrasting workloads: many personalised requests about individual readers, and fewer explorations of connections across a large part of the catalogue.

**E1.** **Scale and performance.** Build three increasingly large subsets of the *real* dataset, the largest containing **all ratings available in the dataset**. Record the number of readers, books and activity records; measure loading time, storage requirements and the execution time of representative services.

**E2.** **Local versus distributed operation under two workloads.** Construct a personalised workload and an exploration workload from the services above. **Before running the measurements**, write down which configuration you expect to perform better under each workload and why, and deposit this prediction on the course platform (or send it to the instructor). Then compare the local deployment and both distributed configurations on the largest dataset of E1, with identical inputs, varying the proportion of the two request types. Report repeated measurements with central tendency and variability, state the cache and warm-up conditions, and discuss the work performed on each instance. A wrong prediction that is well explained is valued as much as a correct one.

**E3.** **Recommendation validity.** Using the evaluation set withheld before any construction, measure whether BookScope recovers relevant books for eligible readers, using a justified metric and a popularity-based baseline. Report results separately for short histories (obtained by truncation, with $k$ stated) and for complete histories. Identify possible information leakage from aggregate metadata or preprocessing, and quantify its effect where possible.

**E4.** **A difficult case (optional).** This experiment is optional and counts as a bonus. Find an actual reader, book or thematic interest for which your modelling or distribution choice gives an unexpectedly slow, weakly supported or misleading result. Explain it, change one design decision and test whether the problem improves. Quantify any negative effect of the change elsewhere in the system.

Distinguish improvements due to better organisation from those caused by a smaller workload or

different input data. Several local server processes still share physical CPU, memory and disk; account for this in your interpretation, and do not assume that adding instances improves performance. Report the hardware and software versions, resource limits and request parameters.

**Independent verification.** Verify at least **two non-trivial results** (one from F2 and one from F4 or F5) with a separate script that reads the original CSV files directly, without using your database.

## 6. Submission and defence

### Submission package

By **Friday, 6 November 2026**, submit: the written report (at most 10 pages, excluding appendices) describing the platform, data preparation, modelling decisions, implementation, experimental protocol, results and limitations; the complete source code and deployment configuration; the raw measurements; and a README with setup and execution instructions allowing the main experiments to be reproduced.

### Defence

Each group gives a **15-minute presentation**, including a live demonstration of at most five minutes, followed by a **10-minute question-and-answer session**. During the questions, the group will face a **live challenge** prepared from its registered selection, for example:

- running services on reader IDs, book IDs and interests chosen by the jury, including a reader whose history is truncated to its first five ratings;
- importing, through F6, a batch of genuine ratings supplied at the defence, then showing that a recommendation changes accordingly;
- tracing a recommendation or a connection back to its source records;
- explaining a file or function of the source code chosen by the jury, by a group member also chosen by the jury.